

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A
DISTANCIA

Universidad Tecnológica Nacional (UTN)

Trabajo Práctico Integrador
Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Materia	Programación 1
Año / Cuatrimestre	2025 – 2° Cuatrimestre
Fecha de entrega	15 de junio de 2026
Integrantes	Juan Gabriel Alejandro Amoroso (Comisión 08)
	Hernán David Ávila (Comisión 09)

Programación 1 – 2C 2025

Índice

1. Marco Teórico

- 1.1. Listas en Python
- 1.2. Diccionarios en Python
- 1.3. Funciones
- 1.4. Estructuras Condicionales
- 1.5. Ordenamientos
- 1.6. Estadísticas Básicas
- 1.7. Archivos CSV

2. Decisiones Técnicas y Arquitectura

- 2.1. Estructura del Proyecto
- 2.2. Diagrama de Flujo
- 2.3. Capturas de Pantalla

3. Dificultades y Conclusiones

4. Bibliografía / Webgrafía

5. Links al Repositorio y Video

1. Marco Teórico

A continuación se explican los conceptos fundamentales de Python utilizados en el desarrollo de este proyecto, con referencias a la documentación oficial.

1.1. Listas en Python

Una **lista** es una estructura de datos mutable y ordenada que permite almacenar una colección de elementos de cualquier tipo. En Python se definen usando corchetes y permiten operaciones de inserción, eliminación, búsqueda y recorrido mediante bucles.

En este proyecto, la variable `países` es una lista de diccionarios donde cada elemento representa un país con sus atributos.

```
países = [] # lista vacía que se completa al leer el CSV
```

Fuente: Python Software Foundation. (2024). Built-in Types – Lists. <https://docs.python.org/3/library/stdtypes.html#lists>

1.2. Diccionarios en Python

Un **diccionario** es una colección no ordenada de pares clave-valor. Permite acceder a los datos mediante claves en lugar de índices numéricos, lo que resulta ideal para representar objetos con atributos nombrados.

Cada país del dataset se representa como un diccionario con las claves `nombre`, `poblacion`, `superficie` y `continente`:

```
pais = {"nombre": "Argentina", "poblacion": 45376763, "superficie": 2780400, "continente": "América"}
```

Fuente: Python Software Foundation. (2024). Built-in Types – Dictionaries. <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

1.3. Funciones

Las **funciones** son bloques de código reutilizables que encapsulan una tarea específica. Se definen con la palabra clave `def` y siguen el principio de *una función = una responsabilidad*, lo que facilita la lectura, el mantenimiento y la prueba del código.

El proyecto sigue una arquitectura modular: el archivo `funciones.py` contiene todas las funciones del sistema, y `main.py` importa y coordina su uso.

```
def cargar_paises(): # Lee el CSV y devuelve una lista de diccionarios
```

Fuente: Python Software Foundation. (2024). Defining Functions. <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

1.4. Estructuras Condicionales

Las **estructuras condicionales** (`if` / `elif` / `else`) permiten que el programa tome decisiones en función del valor de expresiones booleanas. Son fundamentales para validar entradas del usuario, manejar errores y controlar el flujo del menú.

En el sistema se utilizan para verificar que los campos no estén vacíos, que los valores ingresados sean numéricos y que los rangos sean coherentes (mínimo $\leq$ máximo).

Fuente: Python Software Foundation. (2024). More Control Flow Tools. <https://docs.python.org/3/tutorial/controlflow.html>

1.5. Ordenamientos

Python provee la función incorporada `sorted()` que devuelve una nueva lista ordenada sin modificar la original. Acepta un parámetro `key` para definir el criterio de ordenamiento y `reverse` para el sentido (ascendente o descendente).

En el proyecto se utilizan funciones `lambda` como criterio de ordenamiento por nombre, población o superficie:

```
ordenados = sorted(paises, key=lambda p: p['poblacion'], reverse=True)
```

Fuente: Python Software Foundation. (2024). Sorting HOW TO. <https://docs.python.org/3/howto/sorting.html>

1.6. Estadísticas Básicas

El sistema calcula indicadores estadísticos básicos sobre el dataset de países usando funciones incorporadas de Python:

- **max()** / **min()**: obtienen el país con mayor o menor población/superficie.
- **sum()**: suma total de todos los valores de un campo.
- **len()**: cantidad de elementos en la lista.
- **Promedio**: calculado como `sum() // len()`.

Además se agrupa la cantidad de países por continente usando un diccionario como contador.

Fuente: Python Software Foundation. (2024). Built-in Functions. <https://docs.python.org/3/library/functions.html>

1.7. Archivos CSV

Un archivo **CSV** (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos están separados por comas. Python incluye el módulo `csv` en su biblioteca estándar para leer y escribir este tipo de archivos.

El proyecto usa `csv.DictReader` para leer cada fila como un diccionario y `csv.DictWriter` para guardar los cambios:

```
with open('paises.csv', newline='', encoding='utf-8') as f: lector =  
    csv.DictReader(f)
```

Fuente: Python Software Foundation. (2024). csv – CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>

2. Decisiones Técnicas y Arquitectura

2.1. Estructura del Proyecto

El proyecto se organizó en tres archivos principales, siguiendo el principio de separación de responsabilidades:

Archivo	Responsabilidad	Integrante
main.py	Menú principal, bucle de ejecución y coordinación de llamadas	Dan Amoroso
funciones.py	Todas las funciones: carga, búsqueda, filtros, orden, estadísticas	Hernán Ávila
países.csv	Dataset base con 36 países de todos los continentes	Hernán Ávila

2.2. Diagrama de Flujo del Sistema

El siguiente esquema representa el flujo principal de operaciones del programa:

INICIO
Cargar datos desde paises.csv
Mostrar menú principal
Leer opción del usuario
Opción 1: Agregar país → Validar campos → Guardar CSV
Opción 2: Actualizar país → Buscar → Modificar → Guardar CSV
Opción 3: Buscar por nombre → Mostrar resultados
Opción 4/5/6: Filtrar → Mostrar tabla filtrada
Opción 7: Ordenar por criterio → Mostrar tabla ordenada
Opción 8: Calcular estadísticas → Mostrar resultados
Opción 0: FIN

2.3. Capturas de Pantalla – Ejemplos de Ejecución

A continuación se muestran ejemplos representativos de la ejecución del programa en consola.

Ejemplo 1 – Menú principal al iniciar el programa:

Cargando datos... Se cargaron 36 países correctamente.

SISTEMA DE GESTIÓN
DE PAÍSES

1. Agregar un país 2. Actualizar un país 3. Buscar país por nombre 4. Filtrar por

Elegí una opción:

3. Dificultades y Conclusiones

3.1. Dificultades encontradas

- **Sincronización en GitHub:** Al trabajar en equipo por primera vez con un repositorio compartido, surgieron dificultades para coordinar los commits y evitar conflictos entre archivos. Se resolvió estableciendo una división clara de archivos por integrante y usando pull antes de cada sesión de trabajo.
- **Codificación del CSV:** Al abrir el archivo CSV en diferentes sistemas, algunos caracteres especiales (tildes, ñ) no se mostraban correctamente. Se solucionó especificando explícitamente `encoding='utf-8'` en todas las operaciones de lectura y escritura.
- **Validación de entradas:** Asegurar que el programa no fallara ante entradas incorrectas del usuario (texto donde se espera número, campos vacíos, rangos invertidos) requirió el uso cuidadoso de bloques `try/except` y condicionales de validación.
- **Modularización:** Separar correctamente las responsabilidades entre `main.py` y `funciones.py` requirió planificación previa para evitar que una función hiciera demasiadas cosas a la vez.

3.2. Conclusiones

Este trabajo integrador nos permitió poner en práctica de forma conjunta los conceptos centrales de Programación 1: listas, diccionarios, funciones, condicionales, ordenamientos, manejo de archivos CSV y estadísticas básicas.

La experiencia de trabajar con GitHub fue particularmente valiosa: aprendimos a colaborar en un mismo proyecto sin pisarnos el trabajo, a hacer commits descriptivos y a mantener el repositorio organizado.

Comprendimos la importancia de la modularización: un código dividido en funciones con responsabilidades claras es mucho más fácil de entender, probar y mantener que un código monolítico.

Como reflexión final, reconocemos que la planificación previa y la comunicación constante entre los integrantes del equipo fueron clave para completar el proyecto de manera ordenada y en tiempo.

4. Bibliografía / Webgrafía

- [1] Python Software Foundation. (2024). *Built-in Types – Lists*. Recuperado de: <https://docs.python.org/3/library/stdtypes.html#lists>
- [2] Python Software Foundation. (2024). *Built-in Types – Dictionaries*. Recuperado de: <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
- [3] Python Software Foundation. (2024). *Defining Functions*. Recuperado de: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- [4] Python Software Foundation. (2024). *csv – CSV File Reading and Writing*. Recuperado de: <https://docs.python.org/3/library/csv.html>
- [5] Python Software Foundation. (2024). *Sorting HOW TO*. Recuperado de: <https://docs.python.org/3/howto/sorting.html>
- [6] Python Software Foundation. (2024). *Built-in Functions*. Recuperado de: <https://docs.python.org/3/library/functions.html>
- [7] Python Software Foundation. (2024). *Errors and Exceptions*. Recuperado de: <https://docs.python.org/3/tutorial/errors.html>
- [8] Real Python. (2024). *Reading and Writing CSV Files in Python*. Recuperado de: <https://realpython.com/python-csv/>
- [9] Real Python. (2024). *How to Use Python Lambda Functions*. Recuperado de: <https://realpython.com/python-lambda/>
- [10] GitHub Docs. (2024). *Getting started with GitHub Desktop*. Recuperado de: <https://docs.github.com/en/desktop/overview/getting-started-with-github-desktop>

5. Links al Repositorio y Video

Repositorio GitHub

El código fuente completo del proyecto, incluyendo `main.py`, `funciones.py`, `países.csv` y este documento, se encuentra disponible en el siguiente repositorio público:

<https://github.com/Hernanavila98/gestion-paises-python>

Video Demostrativo

La grabación de demostración del sistema (10–15 minutos) donde se muestra el funcionamiento completo del programa en consola, la explicación de la estructura de datos y los flujos principales, se encuentra disponible en:

[Link al video – pendiente de carga en YouTube/Drive]

Nota: Una vez grabado el video, este enlace se actualizará también en el archivo README.md del repositorio de GitHub.